Making MFC faster on nano4 H200 without changing its answer

CS5434 High Performance Computing, Coding-Adv-HW-1

MFC v5.7.0 (`0b3d56c`), the SC26 Student Cluster Competition application

Result

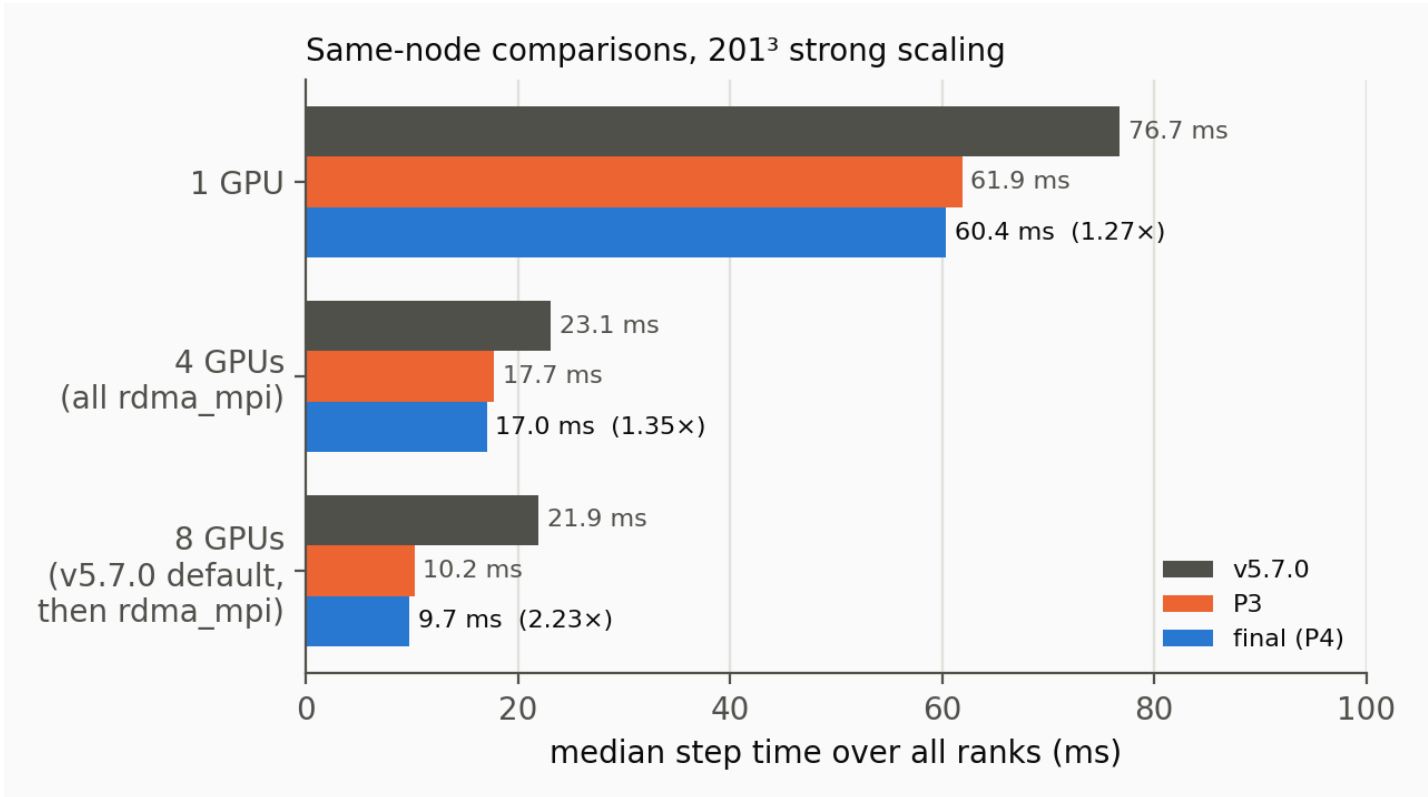
Outputs stay bitwise identical to unmodified v5.7.0.

GPUs	v5.7.0 step	P3 step	Final step	Speedup	v5.7.0 grind	Final grind
1	76.74 ms	61.88 ms	60.38 ms	1.27×	0.3973	0.3123
4 (all `rdma_mpi`)	23.10 ms	17.72 ms	17.03 ms	1.35×	0.1184	0.0876
8 (v5.7.0 default, then `rdma_mpi`)	21.94 ms	10.22 ms	9.75 ms	2.23×	0.1129	0.0502

- Step: median NVTX step time (steps 2–30) of the slowest rank. Grind: ns / grid point / equation / RHS.
- Speedup is the product of two same-node ratios: v5.7.0 vs P3, then P3 vs final (P4).
- Strong-scaling efficiency of the final build: 89 % at 4 GPUs, 77 % at 8 GPUs. v5.7.0 at defaults: 44 % at 8 GPUs.

Jobs 606017 / 621788 (1 GPU), 606018 / 622305 (4 GPUs), 605963 / 606019 / 622306 (8 GPUs). The 8-GPU row compares v5.7.0 defaults with the final configuration. A like-for-like v5.7.0 `rdma_mpi` arm was not run (AGENT_LOG #14).

Same-node step times



Median step time, 201³ strong scaling. Each comparison runs on one node.

Rules we held ourselves to

- SC26 spirit: same equations, schemes, grid, step count and FP64. No loosened tolerances, no changed outputs.
- Gate on every run: the final conservative state (65 M doubles) is bitwise identical to pinned v5.7.0 (`harness/verify.py`).
- Every `src/` change passes `./mfc.sh test` (tolerance 1e-12) with no failure that the unmodified build does not also show.
- Source changes are upstream-PR-able: small FYPP macros, OpenACC and OpenMP paths, compiler-portable.
- Build and run-time settings are disclosed.
- Measurement: warm-up, then ≥3 reps (final: 5), medians, A/B on the same node, `nvidia-smi` clocks and power recorded.
- Negative results are reported.

Environment and method

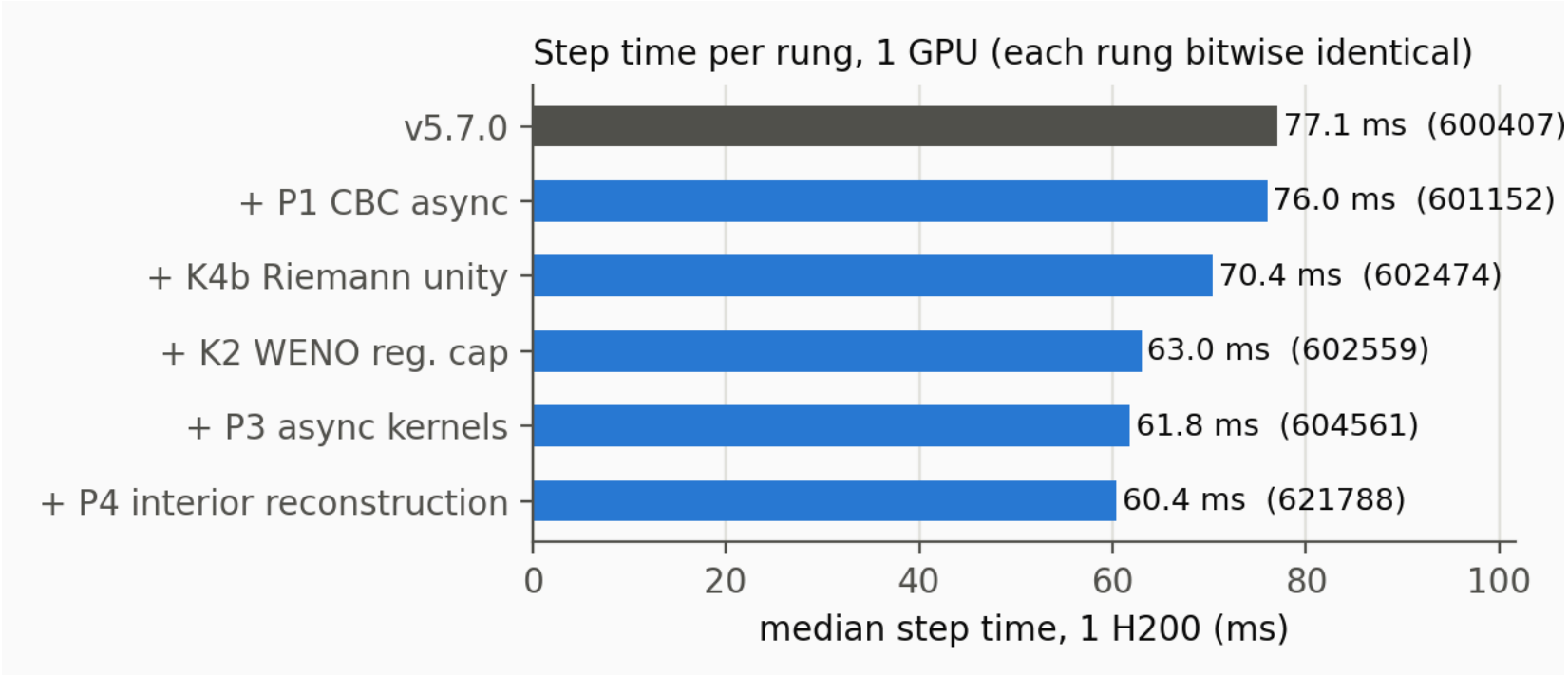
- nano4 compute nodes `25a-hgpn*`: 8× H200 (141 GB, 700 W, NVLink), 2× Xeon 8480+, 10 Mellanox HCAs.
- NVHPC 26.3 (CUDA 13.1, OpenACC), HPC-X 2.25.1 Open MPI 4.1.9, Slurm partition `dev`. One rank per GPU.
- Case: `examples/3D_performance_test`. 5-equation two-fluid model, WENO5-M, HLLC, RK3, FP64, 201^3 cells, 30 steps, `--case-optimization`.
- Harness: `submit.sh` logs every job in `results/jobs.tsv`. `run.sbatch` does warm-up, timed reps and one NVTX-only nsys rep. `ab.sbatch` and `series.sbatch` run several builds on one node.
- Environment fixes before anything ran (no numerics change): `mpirun` instead of `srun`, no OMPI core binding inside the cgroup, ROMIO instead of OMPIO on WekaFS (OMPIO took 337 s or segfaulted).

Baseline: where the time went

- 1 GPU: 77.06 ms per step (600407). GPU busy 96 % of a stage (nsys 600140).
- Kernel split: WENO5 34 %, HLLC 25 %, copy-like kernels ≈25 % at ~30 % of HBM bandwidth (ncu 600146).
- 8 GPUs at defaults: 3.6× over 1 GPU (grind). Halo exchange is 53 % of a stage (host-staged).
- With `rdma_mpi=T`: 5.6×. A rank with more CBC boundary faces spent 0.9 ms per stage more on the host. Everyone else waited for it in `MPI_Sendrecv`.
- GPU work itself was balanced within 2 %.

Plan: first remove host round trips (parallel efficiency), then raise single-GPU kernel efficiency.

Optimization ladder, 1 GPU



Median step time per rung on 1 H200, with Slurm job id. Rungs measured in sequence. Each rung is bitwise identical.

P1 and K4b: host syncs and inlining

P1, CBC without host round trips (`0001` , `m_cbc.fpp` , +50/−44)

- Each characteristic-boundary call issued ~12 small kernels and 5 scalar updates, each with a host sync (~230 µs per call).
- Fix: queue them async, wait once. −1.4 % at 1 GPU, −2.3 % at 4 GPUs (601183 vs 601184).

K4b, let the compiler inline the Riemann helpers (`0002` + `0003` , CMake + 3 lines)

- HLLC was 42 % slower than in v4.9.3. The PTX had 9 `.extern .func` calls.
- nvfortran inlines `acc routine seq` routines only within one source file. Arguments went through local memory.
- Fix: compile the module slice `m_variables_conversion … m_riemann_solvers` as one translation unit.
- Result: 0 calls, local loads 178 → 76, all stores `STG` . HLLC 18.7 → 12.9 ms per step.

K2 and P3: occupancy and async kernels

K2, WENO occupancy (`0004` , one per-file flag, disclosed)

- WENO5 needs 188–196 registers, so 8 warps per SM. Latency-bound, FP64 pipe 31 %.
- `-gpu=maxregcount:128` for `m_weno` alone gives 16 warps per SM. Sweep 168 / 128 / 96 → 64.0 / 63.0 / 66.5 ms.
- WENO5 25.4 → 18.0 ms per step (nsys 602481 → 602705).

P2 + P3, no host round trip per kernel (`0005` , 10 files, +170/−153)

- Each RK stage still launched 44–68 kernels and ~45 scalar `GPU_UPDATE` s with host syncs: ~1.2 ms GPU idle per stage.
- Fix: alias the async queue to the sync stream at start-up, `GPU_ASYNC` on 115 per-stage directives, one `GPU_WAIT` before MPI.
- Other compilers and the OpenMP path generate the old code.
- Same-node A/B: 4 GPUs `rdma_mpi` 18.81 → 17.71 ms (−5.8 %), 1 GPU −1.7 %.

P4: reconstruct only what the Riemann solver reads

`0006`, `m_rhs.fpp` + `m_weno.fpp`, +29/−14

- After P3, WENO5 and its input pack were 45 % of kernel time (ncu 606033).
- Each sweep reconstructed the full transverse ghost range (4 cells per side). The Riemann solver reads only transverse-interior cells.
- An audit of every consumer in `simulation/*.fpp` found no other reader.
- The skipped share grows as boxes shrink: 7.5 % of a sweep at 201^3 per GPU, 14 % at 101^3.

GPUs	P3	P4	Change	Job
1	61.90 ms	60.38 ms	−2.5 %	621788
4	17.65 ms	17.03 ms	−3.5 %	622305
8	10.15 ms	9.75 ms	−4.0 %	622306

Run-time settings (disclosed)

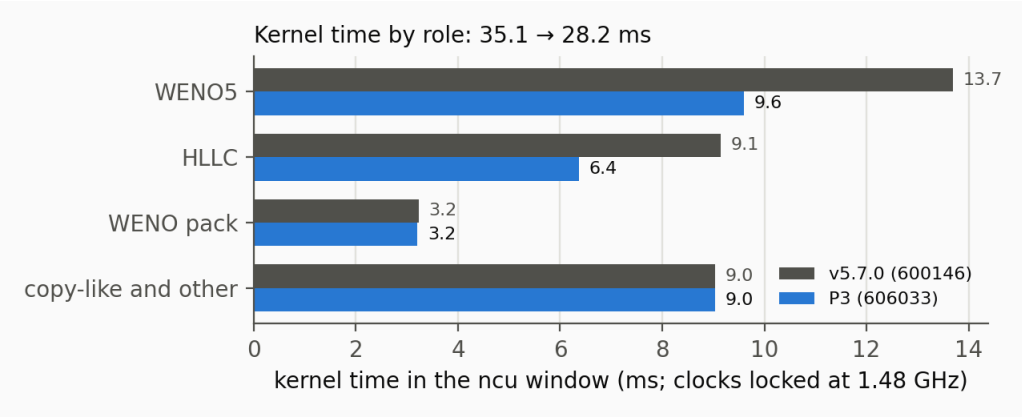
`rdma_mpi=T`, an MFC input parameter

- Halos go GPU→GPU over NVLink (UCX `cuda_ipc`, 292 GB/s measured) instead of through host memory.
- Bitwise. −27 % per step at 4 GPUs (602770 vs 604560, different nodes).

T0.4, MPI start-up

- `MPI_Init` took 1.7 s at 1 rank and 5.8–6.5 s at 4 ranks, because UCX opens all 10 HCAs.
- `harness/nicbind.sh`: each rank uses the HCA next to its GPU.
- With host-staged halos at 4 GPUs it made each step 4.5 % slower, so it was rejected there.
- With `rdma_mpi=T`: step 17.65 → 17.69 ms (noise), wall 13.03 → 9.28 s (−29 %, 606032).

Why it runs at this speed now



ncu 600146 (v5.7.0) vs 606033 (P3), clocks locked at 1.48 GHz

Kernel (per call)	v5.7.0	P3
WENO5	3.39–3.49 ms	2.34–2.56 ms
HLLC	2.13–2.48 ms	1.57–1.62 ms
Copy-like	0.72–1.19 ms	unchanged
All listed	35.10 ms	28.20 ms

- WENO5: occupancy doubled, FP64 pipe 31 % → 42–46 %.
- HLLC: helpers inlined, FP64 pipe 9–10 % → 13 %.
- Copy-like kernels reach a third of HBM bandwidth. The SMs are busy with index arithmetic.

Negative results

Idea	Result	Job
NVHPC 25.9 instead of 26.3	no gain, not bitwise	ladder row 1
Compile-time `dir_idx` in HLLC (K4a)	SASS worse (255 registers)	ladder row 3
WENO `vector_length(64/96)`	registers rose to 206 / 192, not timed	`results/k2/`
WENO register cap 96	spills dominate, 66.5 ms	602567
HCOLL/UCC off	no clear effect, inside control spread	602638/9, 602683
One HCA per rank, host-staged halos	−3 s start-up, +4.5 % per step	602770/1
`ob1` /vader, UCX shm	peers unreachable / no gain	602704/26

Correctness

- Every benchmark run: final state bitwise identical to pinned v5.7.0 (`verify.json` per run).
- Full `./mfc.sh test` on P4: 752 listed, 713 ran, 690 pass, 23 fail, 0 new failures (jobs 622349–622356).
- All 23 failures fail identically on unmodified v5.7.0 in this environment (re-run 606331, final build 606332).
- They are 17 `model_eqns=3` / viscous IBM cases, five examples and one reactive-burn case. List: `harness/known_test_failures.txt`.
- So the final series changes no test outcome. The v5.7.0 suite is not clean on NVHPC 26.3 / H200, and we report that.
- Two harness holes found and closed: `verify.py` once read the warm-up output (AGENT_LOG #12), and 1-GPU test jobs could not start 2-rank tests.

Next steps and reproduction

Next

1. Overlap the halo exchange with interior compute. P3 removed the host syncs that would have eaten the overlap.
2. Fuse or skip the bandwidth-bound copies (40 % of kernel time).
3. Submit P1, K4b, P3 and P4 upstream as small PRs.

Reproduce on nano4

```
bash harness/reproduce.sh build    # v5.7.0 + patches/opt/0001-0006
bash harness/reproduce.sh run      # final 1/4/8-GPU same-node sets
bash harness/reproduce.sh verify   # bitwise gate + median step per run
```

Thank you

Report: `report.md`. Patches: `patches/opt/`. Raw results: `results/`.